

BUILD REPORT

2026-09-09

Signalpost Agent

commit 610ec75 + uncommitted
144 tests, 0 live network
repo: signal-post-builderr

An autonomous pipeline turning a bare Norwegian organisation number into a sourced, verifiable company profile — built for Builderr's Signalpost challenge.

LIVE EVALUATOR SCORE	EST. SCORE, THIS SESSION'S WORK	WEIGHTED EXTERNAL RECALL	COST PER 100-COMPANY BATCH
62.12 / 100	~78 / 100	~38%	\$0.00
Below 65 to qualify	Our estimate, not re-evaluated	Needs ≥60% — still the blocker	Cap is \$10

01 What it does

Builderr's brief covers 411,160 Norwegian companies and scores on four phases: **Resolve** the entity, **Crawl** permitted public sources, **Prove** every claim with a source and timestamp, **Refresh** without ever losing history. Every fact the agent publishes carries one of six evidence states — there's no way to emit a claim without picking one, which is what lets the agent get credit for honestly saying "not found" instead of guessing:

- available
- not_available
- blocked
- ambiguous
- failed
- not_applicable

02 Pipeline

Seven stations, each testable in isolation — none reach into another's internals.

01 Resolve resolve.py

Org number → verified legal name, address, official site. Queries the live Brønnøysundregisteret API or Builderr's frozen universe file.

02 **Crawl** `crawl.py`

Static fetch of the official site, same-domain paths, and sitemap.xml/robots.txt-discovered pages. Follows outbound links to allow-listed ATS platforms only when linked from the official site itself.

03 **Extract** `extract.py`

JSON-LD (Organization, JobPosting, NewsArticle), OpenGraph, canonical links, then Trafilatura text as fallback.

04 **Verify** `verify.py`

Domain provenance + RapidFuzz name match. The one function that enforces the 95%-precision gate — nothing is published without clearing it.

05 **Registry extras** `registry_extras.py`

Leadership roles, filed accounts, sub-units — straight from Brreg's own `roller` / `regnskapsregisteret` / `underenheter` endpoints. No identity risk to gate on.

06 **Assemble** `assemble.py`

Builds the validated seven-section profile. A malformed or under-evidenced profile cannot leave this stage.

07 **Persist** `snapshots.py`

Append-only. Every run is a new row, diffed against the last for material changes — never overwritten.

03 Real-world validation

Ran three full 1,000-company batches against the real Brønnøysundregisteret API. Each one surfaced a genuine bug no unit test had caught — the kind that only shows up at scale, with real data.

COMPANIES	REQUESTS	SPEND	WALL CLOCK	TERMINAL RESULTS
1,000	4,674	\$0.00	14m	1,000 / 1,000

Claiming "available" with no value `assemble.py`

Most Norwegian companies (89% of a random sample) have no website on file. The fallback path was labelling that field `state: available, value: null` — a claim of having verified something that was never there. Every one of the 4 hand-built test fixtures happened to be a company *with* a website, so nothing caught it until 893 real profiles showed the pattern.

Shared SQLite connection across threads `storage/cache.py`

The response cache is shared across every company in a batch, dispatched via `asyncio.to_thread`. A raw `sqlite3.Connection` rejects use from any thread but the one that created it — fine in single-threaded tests, a hard crash under real concurrency.

A malformed server redirect `crawl.py`

One real company's site issued a redirect to `https://.example.no/...` — an empty hostname label, a bug on their end. httpx's redirect handling raised a bare `UnicodeError`, a type the retry logic wasn't watching for, so it escaped the stage boundary entirely for that one company.

Verified facts silently discarded `assemble.py`

Canonical-link and search-discovered official-site facts were extracted and verified correctly, then never actually read back out when building the final claim — the assembler only ever looked at the registry's own field.

04 **Evaluator feedback, and the response**

Builderr's evaluator returned six concrete recommendations after the first submission. All six were implemented this session, test-first.

RECOMMENDATION	STATUS
Read sitemap.xml / robots.txt, prioritise About/Careers/News pages	Done
JSON-LD JobPosting/datePublished, OpenGraph, canonical links	Done
Follow official outbound links to ATS platforms, keep the link chain	Done
Normalise/dedupe URLs, bind results to the right org number	Done
Bounded crawler with retries, backoff, caching	Done — cache existed, was never wired in
Synthesis interface + a usable results view	Done — both were at 0

Effect on real field coverage, same 1,000-company sample, before and after:

FIELD	BEFORE	AFTER
Hiring signals	6 →	59
Dated activity	~0 →	69
Company profiles (LinkedIn etc.)	~0 →	119
Public brand	0 →	9

Legal name, official website, accounts, leaders and workplaces are unchanged — those five come straight from the registry and don't depend on crawling.

Score estimate, by category

Coverage & discovery	13.2 → ~17 / 35
Accuracy & evidence	28.9 → ~29 / 30
Refresh & extensibility	20 → 20 / 20
Synthesis	0 → ~8 / 10

The total score is not the qualifying number. Qualification needs every hard gate cleared, not 65+ points. Coverage ($\geq 21/35$) and weighted external recall ($\geq 60\%$) both estimate as still failing — a hard-gate failure fails the submission regardless of the total.

05 The recall investigation

Weighted external recall is capped by one hard fact: **89% of a random sample of the company universe has no website on file** in the registry at all. No amount of better crawling reaches a company with no domain to crawl. A search-based discovery step (`discovery.py`) was built and tested — DuckDuckGo's Instant Answer API, zero cost, no key required.

CHECK	RESULT
Real companies tested (no website, >50 employees)	15
Candidates found	0
Reason	Wikipedia-infobox lookup, not general web search

Correctly built, safely gated (a discovery hit is independently re-fetched and name-matched before it's ever trusted), and empirically close to a no-op for the population that actually needs it — the companies missing a website are small regional businesses, not Wikipedia-notable ones.

Three alternatives, reviewed

- **Self-hosted SearXNG** — proxies real Google/Bing results. Would work, but it's the same kind of access I already declined for a bot-blocked source earlier in the project, and it adds a container the "one-command" submission now has to remember to start.
- **Paid free-tier search APIs** (Brave, Google Custom Search) — genuine general web search, no notability ceiling. Google's 100/day free allowance lines up almost exactly with the ~89 lookups

one real daily batch would need, at \$0.

- **Wikidata / OpenCorporates** — legitimate, but Wikidata inherits the same notability ceiling as DuckDuckGo, and OpenCorporates' Norwegian data traces back to Brreg itself — the same gap, not a new source.

06 Engineering discipline

TESTS	SOURCE MODULES	LIVE NETWORK IN TESTS	FIXTURE COMPANIES
144	27	0	4

Every stage was built red-then-green: a failing test written against the real contract, confirmed to fail for the right reason, then made to pass. Fixtures are real recorded Brønnøysundregisteret responses, not synthetic data — the four bugs above were all found running the real pipeline against real numbers, not by inspection.

07 Known limitations

- **Per-company cost/runtime** reports as zero in each envelope — the budget governor is one shared counter per chunk on purpose, so exact per-company attribution under concurrency isn't implemented. Accurate aggregate totals are in `run-report.json`.
- **Group/subsidiary labelling** isn't implemented — the precision gate stops a wrong-company publication, but doesn't yet distinguish a parent company's page from the exact subsidiary's.
- **Playwright** (JS rendering) is optional and not installed by default — a JS-shell page degrades to its static content rather than failing.
- **Self-check fixture sample is 4 companies**, smaller than the recommended 10–30, for the reason above.

```
signal-post-builderr · run_batch.py --input entry-companies.jsonl --out results/
```

Report generated 2026-09-09